

INFORME ACADÉMICO INDIVIDUAL

Análisis Crítico de Sistemas Distribuidos

Unidad 1 — Generalidades de los Sistemas Distribuidos
Semana 3 | Período Regular 2025-2026 SPA

Asignatura	Aplicaciones Distribuidas — ISR-701
Unidad	1: Generalidades de los Sistemas Distribuidos
Tipo de actividad	Informe Académico Individual
Semana	3 (18–24 de mayo del 2026)
Nombre del estudiante	
Número de cédula	
Fecha de entrega	

1. Introducción

Los sistemas distribuidos constituyen el paradigma arquitectónico dominante de la infraestructura digital contemporánea. Desde los servicios de streaming que millones de personas usan cotidianamente hasta los sistemas financieros que procesan billones de transacciones al año, la distribución del cómputo sobre redes de computadoras heterogéneas ha pasado de ser una opción arquitectónica a una necesidad inevitable.

El presente informe analiza de manera crítica los fundamentos teóricos de la Unidad 1 de la asignatura Aplicaciones Distribuidas, evaluando sus ventajas, limitaciones y retos desde una perspectiva interdisciplinaria. El análisis va más allá de la enumeración de conceptos: busca identificar tensiones internas en el campo, contraejemplos que cuestionan las definiciones canónicas y reflexiones sobre cómo los conceptos teóricos se manifiestan —con todas sus contradicciones— en sistemas reales de producción.

Metodología del informe: El análisis se estructura en cinco secciones que cubren: la definición y propiedades de los SD, el análisis crítico de sus ventajas y limitaciones, la evaluación del teorema CAP como marco conceptual, la comparación de modelos arquitectónicos, y reflexiones personales sobre la relevancia de estos conceptos para la formación del Ingeniero de Software.

2. Definición y Propiedades de los Sistemas Distribuidos — Análisis Crítico

2.1 Definición Canónica y sus Limitaciones

La definición más citada en la literatura es la de Tanenbaum y Van Steen (2023): "A distributed system is a collection of autonomous computing elements that appears to its users as a single

coherent system." o en español: "Un **sistema** distribuido es un conjunto de elementos de computación autónoma que a sus usuarios se les presenta como un sistema único y coherente."

Esta definición tiene dos componentes: (1) la autonomía de los nodos y (2) la percepción de coherencia por parte del usuario.

Sin embargo, esta definición presenta limitaciones importantes que vale la pena examinar críticamente:

- **El problema de la autonomía:** En los sistemas reales, los nodos raramente son completamente autónomos. En un clúster de Kubernetes, los nodos worker ejecutan exactamente lo que el nodo control planea les ordena. ¿Es Kubernetes un sistema distribuido? Sí, pero los nodos no son "autónomos" en el sentido estricto. La autonomía es un espectro, no una propiedad binaria.
- **El problema de la coherencia percibida:** El objetivo de que el sistema "parezca uno solo" es el principio de transparencia. Pero como demostró el fallo de AWS us-east-1 (2011), esta transparencia se rompe precisamente cuando más se necesita: durante los fallos. La definición describe el estado ideal, no el estado operativo habitual.
- **La omisión de la red:** La definición de Tanenbaum no menciona explícitamente la red, que es el componente que hace todo lo demás posible y también el responsable de las propiedades más problemáticas del sistema (latencia variable, particiones, fallas silenciosas).

Una definición complementaria que aborda estas limitaciones es la de Lamport (atribuida informalmente): "A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable." Esta definición captura algo que la de Tanenbaum omite: la interdependencia no obvia y la propagación impredecible de fallos.

2.2 Las Ocho Transparencias ANSA — Evaluación Crítica

La arquitectura de referencia ANSA define ocho transparencias que un sistema distribuido debería proveer. A continuación se evalúa críticamente cada una, distinguiendo entre el ideal teórico y la realidad operativa:

Transparencia	Definición	Nivel de logro real	Ejemplo de violación
Acceso	Ocultar diferencia entre recursos locales y remotos	Parcial	Latencia de red siempre perceptible en sistemas interactivos
Ubicación	Ocultar dónde está el recurso	Alto	CDN logra esto eficientemente; DNS es el mecanismo universal
Migración	Ocultar que el recurso se movió	Medio	Live migration en VMs funciona; en microservicios puede causar fallos de sesión
Reubicación	Ocultar que el recurso se movió MIENTRAS se usa	Bajo	Muy difícil sin estado compartido; los SDKs como gRPC lo facilitan parcialmente
Replicación	Ocultar que hay múltiples copias del recurso	Medio	Consistencia eventual viola esta transparencia desde la perspectiva del usuario
Concurrencia	Ocultar que el recurso es compartido por múltiples clientes	Alto	Bases de datos con MVCC logran esto eficientemente

Transparencia	Definición	Nivel de logro real	Ejemplo de violación
Fallos	Ocultar los fallos y recuperación del sistema	Bajo	Es el más difícil. Fallos de red son inherentemente parciales e invisibles
Persistencia	Ocultar si el recurso está en memoria o disco	Alto	Sistemas de archivos distribuidos (NFS, HDFS) lo logran bien

La conclusión crítica de este análisis es que las transparencias son propiedades de diseño que se aproximan con mayor o menor éxito, no propiedades que se obtienen automáticamente por distribuir un sistema. La transparencia de fallos es estructuralmente la más difícil de lograr, y es precisamente la que más impacta al usuario final.

3. Ventajas, Limitaciones y Retos — Análisis Equilibrado

3.1 Ventajas Reales con Evidencia Cuantificada

Las ventajas de los sistemas distribuidos no deben enunciarse como verdades absolutas sino respaldarse con evidencia empírica. Se presentan las tres principales con datos verificables:

Escalabilidad horizontal

La escalabilidad horizontal (añadir más nodos en vez de máquinas más potentes) es la ventaja más citada. La evidencia es contundente: Twitter en su pico procesaba 150,000 tweets/segundo; Google Search indexa billones de páginas con latencia de milisegundos. Ninguno de estos sistemas sería posible con arquitectura centralizada.

Dato cuantificado: Kleppmann (2017) reporta que LinkedIn, al migrar de una arquitectura monolítica a una distribuida basada en microservicios y Kafka, redujo el tiempo de despliegue de sistemas de recomendación de semanas a horas, y aumentó la capacidad de procesamiento de eventos de 100,000 a más de 1,000,000 eventos por segundo.

Tolerancia a fallos y alta disponibilidad

Un sistema bien diseñado no tiene un único punto de fallo (SPOF). Amazon EC2 publica un SLA de 99.99% de disponibilidad mensual, lo que equivale a menos de 52 minutos de inactividad al año — imposible de garantizar con hardware centralizado sin costos prohibitivos.

Localidad geográfica

Los SD permiten acercar el cómputo y los datos al usuario, reduciendo la latencia. Netflix opera CDN en más de 1,000 ubicaciones globales. La latencia de streaming desde un servidor centralizado en EEUU a Ecuador sería inaceptable; con CDN se reduce a decenas de milisegundos.

3.2 Limitaciones Fundamentales — Más Allá de los Problemas Resolubles

Las limitaciones de los SD no son solo ingeniería pendiente — algunas son límites teóricos imposibles de superar:

La ley de los 8 + 1 errores de Deutsch (1994)

Peter Deutsch identificó 8 falacias que los ingenieros principiantes asumen sobre las redes: (1) La red es confiable, (2) La latencia es cero, (3) El ancho de banda es infinito, (4) La red es segura, (5) La topología no cambia, (6) Hay un solo administrador, (7) El costo de transporte es cero, (8) La red es homogénea. A esta lista, James Gosling añadió la novena: (9) La red es estable. Ninguna de estas falacias puede eliminarse completamente — solo gestionarse.

El límite de la velocidad de la luz

La latencia de red tiene un límite físico: la velocidad de la luz en fibra óptica (~200,000 km/s). La distancia entre Quito y Nueva York es ~4,600 km, lo que implica una latencia mínima teórica de ~23ms de ida y ~46ms de ida y vuelta. Cualquier sistema que requiera coordinar acciones entre nodos en esas ubicaciones no puede garantizar tiempos menores a ese umbral, independientemente de cuánta ingeniería se invierta.

Complejidad de depuración y observabilidad

El estado de un sistema distribuido está fragmentado entre todos sus nodos. No existe una "fotografía" global del estado del sistema en un instante dado (esto es precisamente lo que el problema del tiempo distribuido de Lamport intenta resolver). Depurar un fallo que involucra 10 microservicios comunicándose asíncronamente es cualitativamente más difícil que depurar un monolito.

4. El Teorema CAP — Análisis Crítico e Interpretación Moderna

4.1 Formulación Original y Prueba

El teorema CAP fue propuesto por Eric Brewer en el año 2000 como conjetura y formalmente probado por Gilbert y Lynch en 2002. El teorema establece que un sistema de almacenamiento distribuido no puede garantizar simultáneamente:

- **Consistencia (C):** Todo nodo ve los mismos datos al mismo tiempo (equivalente a consistencia linealizable).
- **Disponibilidad (A):** Toda solicitud recibe una respuesta (no necesariamente la más reciente).
- **Tolerancia a Particiones (P):** El sistema continúa operando aunque algunos mensajes entre nodos se pierdan o se retrasen indefinidamente.

La prueba de Gilbert y Lynch es constructiva: demuestran por contradicción que si una red puede sufrir una partición (lo cual es inevitable en redes reales), entonces el sistema debe elegir entre C y A durante esa partición.

4.2 Crítica y Revisión — El Modelo PACELC

El teorema CAP, a pesar de su elegancia, ha sido criticado por simplificar en exceso la realidad. Las críticas más sustanciales son:

- **La partición es un evento, no un estado permanente:** El CAP describe qué hacer DURANTE una partición. Pero la mayoría del tiempo los SD operan sin particiones, y en ese estado normal también hay trade-offs que el CAP ignora: ¿qué tan consistente es el sistema durante operación normal? ¿Cuánta latencia adicional agrega mantener la consistencia?
- **"Consistencia" y "Disponibilidad" son propiedades graduales:** El CAP las trata como binarias. En la práctica, hay muchos niveles de consistencia (fuerte, causal, monótona, eventual) y muchos niveles de disponibilidad.

Para abordar estas limitaciones, Abadi (2012) propuso el modelo PACELC: Partition tolerance → choose Availability or Consistency; Else → choose Latency or Consistency. El modelo PACELC añade el trade-off latencia/consistencia durante operación normal, que es donde la mayoría de los sistemas pasan la mayor parte de su tiempo.

Clasificación de sistemas reales según PACELC (Abadi, 2012)

PA/EL (prioridad: Disponibilidad y Latencia): Dynamo (Amazon), Cassandra, Riak. Optimizan para siempre responder rápido, aunque los datos no estén actualizados.

PC/EC (prioridad: Consistencia): HBase, ZooKeeper, BigTable. Nunca responden con datos desactualizados, aunque esto implique mayor latencia o rechazar solicitudes.

PC/EL — Híbrido (el "Santo Grial"): CockroachDB, Google Spanner (con TrueTime). Intentan proveer consistencia fuerte con latencia baja usando relojes atómicos o algoritmos de consenso optimizados. Tienen éxito parcial.

5. Comparación de Modelos Arquitectónicos — Perspectiva Crítica

5.1 Cliente-Servidor vs. Peer-to-Peer — ¿Una Dicotomía Obsoleta?

La distinción entre cliente-servidor y P2P, que domina los libros de texto, resulta cada vez menos útil para describir sistemas modernos. La realidad es que la mayoría de los sistemas contemporáneos son híbridos sofisticados que desafían esta clasificación.

Ejemplo ilustrativo: WhatsApp. La señalización (saber que alguien te llama) ocurre vía servidores centrales de WhatsApp (modelo cliente-servidor). Las llamadas de voz/video van P2P directamente entre dispositivos cuando la red lo permite, y a través de TURN servers cuando no (híbrido). El almacenamiento de mensajes es cliente-servidor. Los backups van a Google Drive/iCloud. Un mismo "sistema" usa tres modelos simultáneamente.

Esto sugiere que la pregunta correcta no es "¿qué modelo usa este sistema?" sino "¿qué modelo usa cada función de este sistema, y por qué?" Los libros de texto presentan los modelos como tipos, cuando en la práctica son componentes que se combinan.

5.2 Rol del Middleware — El Componente Invisible Más Importante

El middleware es probablemente el componente menos estudiado y más crítico de los sistemas distribuidos. La evidencia: el fracaso del middleware propio en Knight Capital (2012) provocó pérdidas de 440 millones de dólares en 45 minutos. Un bug en el middleware de enrutamiento de órdenes de bolsa desencadenó miles de transacciones erróneas que el sistema de trading no detectó porque el middleware de monitoreo tampoco funcionaba correctamente.

Esto ilustra que el middleware no es una capa secundaria de "infraestructura" — es el sistema nervioso del SD. La elección de middleware (qué tipo, qué producto, qué configuración) determina propiedades fundamentales como la garantía de entrega de mensajes, el orden de los eventos y la visibilidad de los fallos.

6. Reflexión Personal — Relevancia para el Ingeniero de Software

La Unidad 1 establece un marco conceptual que, en mi opinión, debería ser la base de cualquier discusión sobre diseño de software en el mundo actual. La realidad es que cualquier sistema de software no trivial que se construya hoy será distribuido, ya sea por diseño (microservicios) o por accidente (la base de datos está en un servidor diferente al servidor web). Esto hace que los conceptos de esta unidad no sean "teoría de sistemas distribuidos" — son simplemente "ingeniería de software aplicada al siglo XXI".

El análisis crítico que este informe ha intentado realizar revela que los conceptos de la unidad son más ricos y más contradictorios de lo que una lectura superficial sugiere. Las transparencias no son

propiedades que se «instalan» — son aproximaciones que se diseñan. El teorema CAP no es una receta de diseño — es un límite teórico que hay que entender para saber cuándo los objetivos de diseño son incompatibles entre sí. Los modelos cliente-servidor y P2P no son opciones mutuamente excluyentes — son vocabulario para describir decisiones parciales dentro de un sistema mayor.

Para mi PFC, el aprendizaje más valioso de esta unidad es la importancia de documentar las decisiones de diseño explícitamente. ¿Es mi sistema CP o AP? ¿Por qué? ¿Qué transparencias estoy prometiendo y cuáles estoy sacrificando? ¿Cuál es mi plan cuando falla el nodo X? Estas preguntas, que antes parecían abstractas, ahora tienen respuestas concretas basadas en fundamentos teóricos verificables.

7. Conclusiones

1. **Las definiciones canónicas son puntos de partida, no puntos de llegada:** La definición de Tanenbaum de sistemas distribuidos describe el ideal; la definición informal de Lamport describe la realidad operativa. Ambas son necesarias para tener una comprensión completa.
2. **Las transparencias ANSA son propiedades de diseño graduales:** La transparencia de fallos es estructuralmente la más difícil y la de mayor impacto en el usuario. Los diseñadores deben priorizar explícitamente qué transparencias garantizan y en qué condiciones.
3. **El teorema CAP es un punto de partida conceptual, no una guía de diseño suficiente:** El modelo PACELC de Abadi (2012) extiende el CAP al caso más común (sin partición) y es más relevante para la toma de decisiones de diseño en sistemas de producción.
4. **Los modelos arquitectónicos son componentes, no tipos:** Los sistemas modernos combinan cliente-servidor, P2P e híbrido en diferentes capas funcionales. La pregunta correcta es qué modelo usar para cada función, no qué modelo es el sistema.
5. **El middleware es el componente de mayor riesgo oculto:** Su fallo puede ser catastrófico (caso Knight Capital) y su éxito es invisible. Debe recibir la misma atención de diseño, prueba y monitoreo que la lógica de negocio.

8. Bibliografía (Norma IEEE)

- [1] M. van Steen and A. S. Tanenbaum, Distributed Systems, 4th ed., ver. 4.03. Maastricht, The Netherlands: Maarten van Steen, Jan. 2025. ISBN: 978-90-815406-3-6. [Online]. Available: <https://www.distributed-systems.net>
- [2] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair, Distributed Systems: Concepts and Design, 5th ed. Boston, MA, USA: Addison-Wesley, 2012. ISBN: 978-0-13-214301-1.
- [3] E. Brewer, "CAP twelve years later: How the 'rules' have changed," IEEE Comput., vol. 45, no. 2, pp. 23-29, Feb. 2012. doi: 10.1109/MC.2012.37
- [4] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," ACM SIGACT News, vol. 33, no. 2, pp. 51-59, Jun. 2002. doi: 10.1145/564585.564601
- [5] D. J. Abadi, "Consistency tradeoffs in modern distributed database system design: CAP is only part of the story," IEEE Comput., vol. 45, no. 2, pp. 37-42, Feb. 2012. doi: 10.1109/MC.2012.33
- [6] M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA, USA: O'Reilly Media, 2017. ISBN: 978-1-4493-7332-0.
- [7] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," Commun. ACM, vol. 21, no. 7, pp. 558-565, Jul. 1978. doi: 10.1145/359545.359563

[8] E. Brewer, "Towards robust distributed systems," in Proc. 19th ACM Symp. Principles Distributed Computing (PODC), Portland, OR, USA, 2000. doi: 10.1145/343477.343502